CS 2XY3 Midterm Exam (Solutions)

Question 1 Solutions

(a) Three OS responsibilities: (1) Process management - creating, scheduling, terminating processes. (2) Memory management - allocating/deallocating memory, virtual memory. (3) File system management - creating, deleting, organizing files and directories.

(b) Monolithic kernel: entire OS in one large binary in kernel space. Advantage: fast (no IPC overhead for internal calls). Disadvantage: a bug in any component can crash the whole system. Microkernel: minimal kernel with services in user space. Advantage: better fault isolation (a crashed driver doesn't crash the kernel). Disadvantage: IPC overhead for service communication.

(c) A system call is the programmatic interface between a user process and the OS kernel. Examples: open() to open a file, fork() to create a new process.

Question 2 Solutions

(a) SJF order: P4(1), P2(3), P5(5), P3(7), P1(10). Waiting times: P4=0, P2=1, P5=4, P3=9, P1=16. Average = (0+1+4+9+16)/5 = 6.0 ms.

(b) Round Robin (q=4): P1(4)|P2(3)|P3(4)|P4(1)|P5(4)|P1(4)|P3(3)|P5(1)|P1(2). Waiting: P1=16, P2=4, P3=14, P4=9, P5=13. Average = (16+4+14+9+13)/5 = 11.2 ms.

(c) No. RR does not always outperform FCFS. When all processes have similar burst times, RR introduces context switch overhead without benefit. FCFS has no preemption overhead. RR excels with varied burst lengths where short jobs shouldn't wait behind long ones.

Question 3 Solutions

(a) Virtual pages = 2^32 / 2^12 = 2^20 = 1,048,576 pages.

(b) Physical frames = 512 MB / 4 KB = 2^29 / 2^12 = 2^17 = 131,072 frames.

(c) Each PTE needs: 17 bits (frame number for 2^17 frames) + 8 bits (protection) = 25 bits, rounded to 4 bytes. Table size = 2^20 entries x 4 bytes = 4 MB.

(d) Multi-level preferred because most processes use only a small portion of their address space. A single-level table wastes 4 MB per process even if only a few pages are used. Multi-level allocates inner tables on demand.

Question 4 Solutions

(a) A race condition occurs when the outcome depends on the non-deterministic timing of concurrent operations on shared data. Example: two threads both read counter=5, both increment to 6, both write 6. Result is 6 instead of 7 - one increment is lost.

(b) Bounded buffer with semaphores:

semaphore mutex = 1 (binary)

semaphore empty = N (counting, init to buffer size)
semaphore full = 0 (counting)

Producer: wait(empty); wait(mutex); [add item]; signal(mutex); signal(full)
Consumer: wait(full); wait(mutex); [remove item]; signal(mutex); signal(empty)

(c) Priority inversion: a high-priority task is indirectly blocked by a low-priority task holding a resource it needs, while medium-priority tasks preempt the low-priority task. Mitigation: priority inheritance protocol - temporarily raise the low-priority task's priority to that of the highest-priority task waiting for its resource.